

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to

solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I LATISHA S (192524071) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SOLUTIONS

1. Backtracking Search – Doctor Shift Assignment

We have 3 doctors:

- D1
- D2
- D3

and 3 shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints

1. D1 cannot work Night $\rightarrow D1 \neq N$
2. D2 must work before D3 $\rightarrow \text{Position}(D2) < \text{Position}(D3)$
3. Only one doctor per shift
4. D3 cannot work Morning $\rightarrow D3 \neq M$
5. Every doctor must get exactly one shift

We use Backtracking Search, which assigns values one by one and checks constraints at each step. If a constraint is violated, we reject that assignment and backtrack.

Step-by-step Backtracking

Let us assign shifts in the order $D1 \rightarrow D2 \rightarrow D3$.

Step 1: Assign D1

Try:

D1 = Morning

Check constraints:

- D1 is not Night $\rightarrow \checkmark$
- No other doctor is assigned yet $\rightarrow \checkmark$

So:

D1 = M

Current assignment:

$\{D1 = M\}$

Step 2: Assign D2

Try:

D2 = Morning

Constraint check:

- Morning is already occupied by D1 $\rightarrow \times$
- Only one doctor per shift is allowed.

Therefore, reject:

D2 = M $\rightarrow$ Backtrack

Try:

D2 = Afternoon

Check:

- Afternoon is free $\rightarrow \checkmark$
- D2 can work Afternoon $\rightarrow \checkmark$

So:

D2 = A

Current assignment:

$\{D1 = M, D2 = A\}$

Step 3: Assign D3

Try:

D3 = Morning

Check:

- Morning is occupied by D1 $\rightarrow \times$
- D3 cannot work Morning $\rightarrow \times$

Reject:

D3 = M $\rightarrow$ Backtrack

Try:

D3 = Afternoon

Check:

- Afternoon is occupied by D2 $\rightarrow X$

Reject:

D3 = A $\rightarrow$ Backtrack

Try:

D3 = Night

Check:

- Night is free $\rightarrow \checkmark$
- D3 is not Morning $\rightarrow \checkmark$
- D2 = Afternoon comes before D3 = Night $\rightarrow \checkmark$
- Every doctor has exactly one shift $\rightarrow \checkmark$

Therefore:

D3 = N

Final Assignment

Doctor	Shift	Constraint Check
D1	Morning	D1 is not Night $\checkmark$
D2	Afternoon	D2 works before D3 $\checkmark$
D3	Night	D3 is not Morning $\checkmark$
All	Different shifts	One doctor per shift $\checkmark$

Final Valid Schedule

Morning $\rightarrow$ D1

Afternoon $\rightarrow$ D2

Night $\rightarrow$ D3

Therefore, the valid assignment is:

{D1 = Morning, D2 = Afternoon, D3 = Night}

The backtracking algorithm rejected invalid choices and continued until it found an assignment satisfying all constraints.

2. Robot Grid Search Using BFS and Manhattan Distance

A robot moves in a 5×5 grid from Start (S) to Goal (G).

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Obstacles cannot be crossed
- Manhattan Distance:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

Grid

	1	2	3	4	5
1	S	.	#	.	.
2	#	.	#	.	#
3	.	.	.	.	.
4	.	#	#	.	#
5	.	.	.	.	G

Where:

- S = (1,1)
- G = (5,5)
- # = Obstacle
- . = Free cell

Manhattan Distance

The Manhattan distance is:

$$h(n) = |x_n - 5| + |y_n - 5|$$

For the starting node:

$$h(S) = |1 - 5| + |1 - 5| = 8$$

The Manhattan distance gives an estimate of how far a node is from the goal.

Note: Standard BFS uses a FIFO queue, not a priority queue. Manhattan Distance is normally used with A* or Greedy Best-First Search. Since all moves have cost 1, BFS can find the optimal shortest path.

BFS Node Expansion

BFS explores nodes level by level.

Step 1

Start:

(1,1)

Possible moves:

- Down $\rightarrow$ (2,1) = Obstacle $\times$
- Right $\rightarrow$ (1,2) = Free $\checkmark$

Queue:

[(1,2)]

Step 2

Expand (1,2).

Possible moves:

- Down $\rightarrow$ (2,2) $\checkmark$
- Left $\rightarrow$ (1,1) Already visited
- Right $\rightarrow$ (1,3) Obstacle $\times$

Queue:

[(2,2)]

Step 3

Expand (2,2).

Possible moves:

- Down $\rightarrow$ (3,2) $\checkmark$
- Left $\rightarrow$ (2,1) Obstacle $\times$
- Right $\rightarrow$ (2,3) Obstacle $\times$

Queue:

[(3,2)]

Step 4

Expand (3,2).

Possible moves:

- Left $\rightarrow (3,1) \checkmark$
- Right $\rightarrow (3,3) \checkmark$

Queue:

[(3,1),(3,3)]

Step 5

Expand (3,1).

Possible move:

- Down $\rightarrow (4,1) \checkmark$

Queue:

[(3,3),(4,1)]

Step 6

Expand (3,3).

Possible move:

- Right $\rightarrow (3,4) \checkmark$

Queue:

[(4,1),(3,4)]

Step 7

Expand (4,1).

Possible move:

- Down $\rightarrow (5,1) \checkmark$

Queue:

[(3,4),(5,1)]

Step 8

Expand (3,4).

Possible moves:

- Up $\rightarrow (2,4) \checkmark$

- Down $\rightarrow (4,4)$ ✓
- Right $\rightarrow (3,5)$ ✓

Queue:

$[(5,1),(2,4),(4,4),(3,5)]$

Step 9

Expand (5,1).

Possible move:

- Right $\rightarrow (5,2)$ ✓

Queue:

$[(2,4),(4,4),(3,5),(5,2)]$

Step 10

Expand (2,4).

Possible move:

- Up $\rightarrow (1,4)$ ✓

Queue:

$[(4,4),(3,5),(5,2),(1,4)]$

Step 11

Expand (4,4).

Possible move:

- Down $\rightarrow (5,4)$ ✓

Queue:

$[(3,5),(5,2),(1,4),(5,4)]$

Step 12

Expand (3,5).

All possible directions are either obstacles, outside the grid, or already visited.

No new node.

Queue:

$[(5,2),(1,4),(5,4)]$

Step 13

Expand (5,2).

Possible move:

- Right $\rightarrow$ (5,3) ✓

Queue:

[(1,4),(5,4),(5,3)]

Step 14

Expand (1,4).

Possible move:

- Right $\rightarrow$ (1,5) ✓

Queue:

[(5,4),(5,3),(1,5)]

Step 15

Expand (5,4).

Possible move:

- Right $\rightarrow$ (5,5) = Goal ✓

Goal reached.

BFS Search Summary

3. Autonomous Rescue Robot – Online Search in a Partially Observable Environment

(a) Analysis of Constraints and Their Effect on Decision-Making

The rescue robot operates in a partially observable environment. It does not initially know the complete state of the building and can only observe nearby cells.

1. Movement Constraints

The robot can move:

- Up
- Down
- Left
- Right

Therefore, diagonal movement is not allowed. At every step, the robot must choose one of these four actions.

2. Obstacles

Some rooms or paths are blocked.

The robot must:

- Detect obstacles using its sensors.
- Avoid blocked cells.
- Update its internal map when new obstacles are discovered.

Therefore, the robot cannot simply follow a fixed path based on an initially known map.

3. Limited Sensing

The robot can only observe adjacent cells.

This means the robot has incomplete information about the environment.

It must:

1. Observe the neighboring cells.
2. Update its knowledge.
3. Select the next action.
4. Move to the new cell.
5. Sense again.
6. Re-plan if necessary.

This is called an online search process.

4. Movement Cost

Each normal movement costs:

1

The robot tries to minimize the total movement cost.

5. Risky Zones

Entering a risky zone adds an extra cost of:

2

Therefore, if a normal movement costs 1:

$$\text{Cost of entering risky zone} = 1 + 2 = 3$$

The robot should prefer a safe route if it does not result in an unnecessarily large total cost.

6. Dynamic Knowledge

The robot does not know everything in advance. Its knowledge changes as it explores.

For example:

Initial Knowledge → Sense → Discover Obstacle → Update Map → Recalculate Path

Thus, the robot must continuously make decisions based on its current knowledge.

(b) Solution Approach: Uniform Cost Search + Online Search Agent

A suitable solution is an Online Search Agent using Uniform Cost Search (UCS).

UCS is suitable because different actions have different costs:

- Normal cell: cost = 1
- Risky cell: cost = 3

The robot should select the path with the lowest accumulated cost.

Step 1: Initialize

The robot starts at:

S

The goal is:

G

The robot initially knows only:

- Its current location.
- Information obtained from its sensors.
- Previously explored cells.

Step 2: Sense the Environment

The robot observes adjacent cells.

For example:

S → Observe → Free, Free, Obstacle, Risky

The robot updates its internal map.

Step 3: Generate Possible Actions

The robot considers:

- Up
- Down

- Left
- Right

It removes actions that:

- Go outside the building.
- Enter known obstacles.

Step 4: Calculate Path Costs

For each possible path, calculate the accumulated cost.

For example:

Path A:

5 normal movements

$$5 \times 1 = 5$$

Total cost:

$$5$$

Path B:

3 normal movements + 1 risky movement

$$3(1) + 1(3) = 6$$

UCS prefers Path A because:

$$5 < 6$$

Step 5: Select the Lowest-Cost Node

UCS maintains a priority queue ordered by:

$$g(n) = \text{cost from Start to node } n$$

The node with the smallest accumulated cost is expanded first.

Step 6: Update Knowledge

After every movement:

1. The robot senses adjacent cells.

2. New obstacles are added to the map.
3. Newly discovered risky areas are identified.
4. The current path is checked.
5. If necessary, the robot replans.

Step 7: Reach the Goal

The robot continues exploring and replanning until it reaches:

G

The final solution is the lowest-cost safe path discovered by the robot.

Online Search Process

Sense → Update Map → Plan → Move → Sense Again

This cycle continues until the survivor is reached.

(c) AI Concepts Applied

1. Intelligent Agent

The rescue robot is an intelligent agent because it:

- Perceives the environment through sensors.
- Maintains knowledge about the environment.
- Makes decisions.
- Performs actions.
- Learns new information from exploration.
- Changes its plan when the environment changes.

The agent can be represented as:

Sensors → Agent → Actions

For example:

Sensor: Detects an obstacle

Agent: Updates map and recalculates route

Action: Chooses a different direction

2. Rationality

A rational agent chooses the action that is expected to produce the best result based on the information currently available.

In this problem, the rational robot should:

- Avoid obstacles.
- Avoid unnecessary risky zones.
- Minimize total cost.
- Reach the survivor safely.
- Re-plan when new information becomes available.

The robot does not necessarily choose the shortest path in terms of the number of steps. Instead, it chooses the path with the minimum total cost.

For example:

- Route A: 8 normal moves $\rightarrow$ Cost = 8
- Route B: 5 normal moves + 2 risky moves $\rightarrow$ Cost = $5 + 2(3) = 11$

The rational robot selects Route A because:

$$8 < 11$$

3. Environment Type

The rescue environment can be classified as follows:

Environment Property	Classification	Reason
Observability	Partially Observable	Robot sees only adjacent cells
Determinism	Initially uncertain / potentially nondeterministic	The robot does not know the complete environment
Episodic/Sequential	Sequential	Current actions affect future decisions
Static/Dynamic	Dynamic or changing	New information and conditions may appear
Discrete/Continuous	Discrete	Grid consists of individual cells

Environment Property	Classification	Reason
Number of Agents	Primarily Single-Agent	The robot is the main decision-making agent
Known/Unknown	Initially Unknown	Full map is not available

Therefore, this is primarily a partially observable, sequential, dynamic, discrete, initially unknown environment.

(d) Why Uniform Cost Search + Online Search Is Suitable

The chosen approach is suitable for the following reasons:

1. Different Costs

Normal movement costs 1, while risky zones have an additional cost of 2.

Therefore, the robot cannot use ordinary BFS because BFS assumes all actions have equal cost.

UCS handles different action costs effectively.

2. Minimum-Cost Solution

UCS expands the node with the lowest accumulated path cost.

Therefore, when costs are non-negative, it can find an optimal minimum-cost path based on the robot's current map.

3. Partial Observability

The robot does not have complete information.

An online search agent allows the robot to:

- Explore the environment.
- Gather new information.
- Update its knowledge.
- Re-plan its route.

4. Obstacle Avoidance

When the robot discovers a blocked cell, it can remove that cell from its available paths and search for an alternative route.

5. Safe Navigation

Risky zones have additional costs, so the search naturally considers their penalty.

The robot can select a safer route when it is cheaper or when the problem's safety policy requires avoiding risk.

6. Dynamic Replanning

If the robot's current planned path becomes unavailable, it can perform another search from its current position.

Thus:

UCS + Online Search Agent = Cost-Aware and Adaptive Navigation
--